

SymPy Bug Report

1 Bug 52: `function_range` Says `sqrt(tan(x)**2)` Is Constant

1.1 Minimal Reproducer

```
from sympy import Interval, Rational, N, sqrt, tan, symbols
from sympy.calculus.util import function_range

x = symbols("x")
expr = sqrt(tan(x)**2)
actual = function_range(expr, x, Interval(-1, 1))

print("function_range =", actual)
print("value_at_0 =", expr.subs(x, 0))
print("value_at_1_over_2 =", N(expr.subs(x, Rational(1, 2)), 50))
print("range_contains_0 =", actual.contains(0))
print("expected_range =", Interval(0, tan(1)))
```

1.2 Observed Behavior

```
function_range = {tan(1)}
value_at_0 = 0
value_at_1_over_2 = 0.54630248984379051325517946578028538329755172017979
range_contains_0 = False
expected_range = Interval(0, tan(1))
```

SymPy returns the singleton set $\{\tan(1)\}$. This says the expression is constant on $[-1, 1]$, but the same expression evaluates to 0 at $x = 0$ and to a positive interior value at $x = 1/2$. Thus the returned set omits real values attained by the input expression.

1.3 Expected Behavior

The mathematically correct result is

$$\text{function_range}\left(\sqrt{\tan(x)^2}, x, [-1, 1]\right) = [0, \tan(1)].$$

In SymPy notation, the expected output is `Interval(0, tan(1))`.

1.4 Mathematical Explanation

The interval $[-1, 1]$ lies strictly inside the principal continuity interval $(-\pi/2, \pi/2)$ of $\tan(x)$. On this interval, $\tan(x)$ is continuous and strictly increasing, with $\tan(0) = 0$, $\tan(-1) < 0$, and $\tan(1) > 0$. For real x ,

$$\sqrt{\tan(x)^2} = |\tan(x)|.$$

Therefore the minimum is attained at the interior point $x = 0$, where the value is 0, and the maximum is attained at the endpoints, where

$$|\tan(-1)| = \tan(1), \quad |\tan(1)| = \tan(1).$$

The range is consequently the full interval $[0, \tan(1)]$. The singleton $\{\tan(1)\}$ is not an equivalent representation, because it excludes the attained value 0 and every interior value such as $|\tan(1/2)|$.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation as `sympy/calculus/util.py`, in the public `function_range` routine. The traced call path enters `function_range` around line 161, computes the continuous domain with `continuous_domain` around line 233, then in the interval branch collects endpoint values around lines 253–258 and critical points from

```
solveset(f.diff(symbol), symbol, interval)
```

around line 260. The final range is then constructed from the infimum and supremum of the collected values around line 281.

For this input, `continuous_domain` returns $[-1, 1]$, so the expression itself is continuous on the requested interval. However, its derivative is

$$\frac{(2\tan(x)^2 + 2)\sqrt{\tan(x)^2}}{2\tan(x)},$$

which is undefined at $x = 0$. The derivative-zero solve over $[-1, 1]$ returns the empty set, and the implementation does not also include finite points where the derivative is undefined while the original expression remains defined and continuous. As a result, the only sampled values are the two endpoint values $f(-1) = \tan(1)$ and $f(1) = \tan(1)$. The interior cusp value $f(0) = 0$ is never sampled, so the range degenerates to $\{\tan(1)\}$.

A plausible fix direction supported by the diagnosis is to include finite interior points where `f.diff(symbol)` is undefined but `f` is defined and continuous. For this example, the zero of the derivative denominator $\tan(x)$ would add $x = 0$ as a critical point.

1.6 Additional Failing Instances

The same failure occurs for symmetric intervals $[-b, b]$ contained in $(-\pi/2, \pi/2)$. In each case,

$$\sqrt{\tan(x)^2} = |\tan(x)|,$$

so the minimum remains 0 at the cusp $x = 0$, while the endpoint values are both $\tan(b)$. SymPy instead returns only the endpoint value because it misses the derivative-undefined but continuous critical point at the origin.

Input / Expression	SymPy behavior	Expected behavior
$\sqrt{\tan(x)^2}$ on $[-1, 1]$	$\{\tan(1)\}$	$[0, \tan(1)]$
$\sqrt{\tan(x)^2}$ on $[-1/2, 1/2]$	$\{\tan(1/2)\}$	$[0, \tan(1/2)]$
$\sqrt{\tan(x)^2}$ on $[-1/3, 1/3]$	$\{\tan(1/3)\}$	$[0, \tan(1/3)]$
$\sqrt{\tan(x)^2}$ on $[-2/3, 2/3]$	$\{\tan(2/3)\}$	$[0, \tan(2/3)]$
$\sqrt{\tan(x)^2}$ on $[-3/4, 3/4]$	$\{\tan(3/4)\}$	$[0, \tan(3/4)]$
$\sqrt{\tan(x)^2}$ on $[-1/4, 1/4]$	$\{\tan(1/4)\}$	$[0, \tan(1/4)]$

1.7 Related Issues Assessment

The best-effort deduplication check found related public reports involving `function_range`, absolute values, trigonometric expressions, and fragile critical-point handling. The closest matches include a `TypeError` tied to a `ConditionSet` for an expression involving `Abs`, a separate inverse-trigonometric sign or singularity issue, and a slow `function_range` case for a tangent expression. The available evidence did not identify a public report for $\sqrt{\tan(x)^2}$ on $[-1, 1]$, the wrong singleton output $\{\tan(1)\}$, or the missed continuous cusp value 0. The verdict is therefore a new instance of a known failure mode: the general fragility is related to known behavior, but this exact reproducible case is previously unreported and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Interval, Rational, S, sqrt, tan, symbols
from sympy.calculus.util import function_range

@pytest.mark.parametrize(
    "bound",
    [S.One, Rational(1, 2), Rational(1, 3), Rational(2, 3), Rational(3, 4), Rational(1, 4)],
)
def test_function_range_sqrt_tan_squared_includes_cusp_value(bound):
    x = symbols("x")
    assert function_range(sqrt(tan(x)**2), x, Interval(-bound, bound)) == Interval(0, tan(bound))
```